Kontrol Motor DC dan LED dengan PWM via Serial (ESP32)

Dosen Pengampu: Prof. Dr. Basuki Rahmat, S.Si., M.T.

Disusun Oleh:

Nama : Sanafa Mawaddah R. Z

NPM : 24081010153

PROGRAM STUDI INFORMATIKA FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR

2026

A. Tujuan Modul

1. Memahami Konsep PWM: Mengimplementasikan Pulse Width Modulation (PWM) untuk mengatur kecepatan putaran Motor DC dan intensitas nyala LED.
2. Komunikasi Serial: Merancang sistem kontrol perangkat keras yang menerima instruksi melalui antarmuka komunikasi Serial.
3. Penerapan API ESP32 Terbaru: Memahami dan mengaplikasikan sintaks kontrol PWM format terbaru pada library ESP32 Board v3.0+ (penggunaan ledcAttach dan ledcWrite berbasis pin, bukan channel).
4. Integrasi Mikrokontroler dan Motor Driver: Memahami logika perakitan pin kontrol (IN1, IN2, dan ENA) dari mikrokontroler menuju Motor Driver (seperti L298N atau L293D).

B. Skema Rangkaian

Berdasarkan deklarasi pin pada program, skema rangkaian yang digunakan adalah sebagai berikut:

LED Indikator

- Pin Positif (Anoda) LED dihubungkan ke Pin 2 ESP32 (biasanya bisa memanfaatkan LED *built-in* board).
- Pin Negatif (Katoda) LED dihubungkan ke GND (melalui resistor secukupnya).

Motor Driver (misal L298N)

- IN1 dihubungkan ke Pin 27 ESP32.
- IN2 dihubungkan ke Pin 26 ESP32.
- ENA (Enable A untuk kontrol PWM) dihubungkan ke Pin 12 ESP32.
- Output Motor Driver (OUT1 & OUT2) dihubungkan ke kutub Motor DC.
- *Power Supply* Eksternal dihubungkan ke pin VCC/12V Motor Driver, dan GND Motor Driver wajib di-common (digabung) dengan GND ESP32.

C. Konsep & Logika Sistem

Sistem ini beroperasi berdasarkan instruksi teks yang dikirim melalui *Serial Monitor*. ESP32 akan membaca karakter yang masuk dan menyimpannya dalam sebuah *buffer string* hingga mendeteksi karakter *newline* (\n) atau *carriage return* (\r). Setelah instruksi utuh diterima, ESP32 memparsing perintah tersebut untuk mengubah nilai dutyCycle (rentang 0-255 karena menggunakan resolusi 8-bit). Nilai duty cycle ini kemudian diterapkan ke pin ENA (Pin 12) menggunakan sinyal PWM, yang secara

langsung akan mengatur kecepatan putaran motor DC. Motor selalu diset berputar ke satu arah secara *hardcode* (IN1 = HIGH, IN2 = LOW). Sebagai indikator visual tambahan, LED pada Pin 2 akan menyala setiap kali motor bergerak (dutyCycle > 0).

D. Daftar Perintah

Sistem dapat merespon input Serial berikut:

- 1 : Mengaktifkan motor menggunakan nilai kecepatan (duty cycle) terakhir, atau nilai default (200) jika sebelumnya 0.
- 0 : Mematikan motor secara total (duty cycle = 0).
- + : Menaikkan kecepatan motor secara bertahap dengan langkah penambahan 10 (maksimal 255).
- - : Menurunkan kecepatan motor secara bertahap dengan langkah pengurangan 10 (minimal 0).
- PWM:xxx : Mengatur kecepatan motor secara spesifik ke nilai xxx (contoh: PWM:150).

E. Source Code dan Penjelasan Logika Program

```
// Kontrol Motor DC + LED dengan PWM via Serial
// Perintah:
//   '1'       -> Motor ON (duty terakhir / default)
//   '0'       -> Motor OFF
//   '+'       -> Naikkan kecepatan (+10)
//   '-'       -> Turunkan kecepatan (-10)
//   'PWM:xxx'  -> Set duty cycle langsung (0-255)
//
============================================================
=

int ledPin = 2;
char myData = 0;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// Setting PWM properties
const int freq = 30000;
// const int pwmChannel = 0; // <-- Sudah tidak dipakai di ESP32 v3.0+
const int resolution = 8;
int dutyCycle = 200;         // duty cycle awal
const int DUTY_STEP = 10;    // langkah naik/turun
const int DUTY_MIN  = 0;
const int DUTY_MAX  = 255;
```

```
String inputString = "";     // buffer untuk perintah "PWM:xxx"

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  // Konfigurasi PWM format BARU (ESP32 Board v3.0+)
  ledcAttach(enable1Pin, freq, resolution);

  // pastikan motor mati saat start
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  // Format BARU: ledcWrite menggunakan nama pin, bukan channel
  ledcWrite(enable1Pin, 0);

  Serial.println("ESP32 Motor Controller siap.");
  Serial.println("Perintah: 1=ON, 0=OFF, +=naik, -=turun, PWM:xxx=set");
}

void applyMotor()
{
  // Terapkan dutyCycle ke driver motor
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  // Format BARU: menggunakan enable1Pin
  ledcWrite(enable1Pin, dutyCycle);

  // LED indikator: nyala kalau motor bergerak
  digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW);
}

void MotorOn()
{
  // Kalau duty masih 0, pakai default 200
  if (dutyCycle == 0) dutyCycle = 200;
  applyMotor();
  Serial.print("Motor ON, duty = ");
  Serial.println(dutyCycle);
}

void MotorOff()
{
```

```
  dutyCycle = 0;
  applyMotor();
  Serial.println("Motor OFF, duty = 0");
}

void naikkanKecepatan()
{
  dutyCycle += DUTY_STEP;
  if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX;
  applyMotor();
  Serial.print("Kecepatan NAIK, duty = ");
  Serial.println(dutyCycle);
}

void turunkanKecepatan()
{
  dutyCycle -= DUTY_STEP;
  if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN;
  applyMotor();
  Serial.print("Kecepatan TURUN, duty = ");
  Serial.println(dutyCycle);
}

void setDuty(int val)
{
  if (val < DUTY_MIN) val = DUTY_MIN;
  if (val > DUTY_MAX) val = DUTY_MAX;
  dutyCycle = val;
  applyMotor();
  Serial.print("Set duty = ");
  Serial.println(dutyCycle);
}

void loop()
{
  while (Serial.available() > 0)
  {
    char c = Serial.read();

    // Cek perintah "PWM:xxx"
    if (c == '\n' || c == '\r')
    {
      inputString.trim();
      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();
        setDuty(val);
      }
      else if (inputString.length() > 0)
```

```
    {
     // perintah satu karakter
     char cmd = inputString.charAt(0);
     if (cmd == '1')      MotorOn();
     else if (cmd == '0') MotorOff();
     else if (cmd == '+') naikkanKecepatan();
     else if (cmd == '-') turunkanKecepatan();
     else {
       Serial.print("Perintah tidak dikenal: ");
       Serial.println(inputString);
     }
    }
    inputString = "";
  }
  else
  {
    inputString += c;
  }
}
```

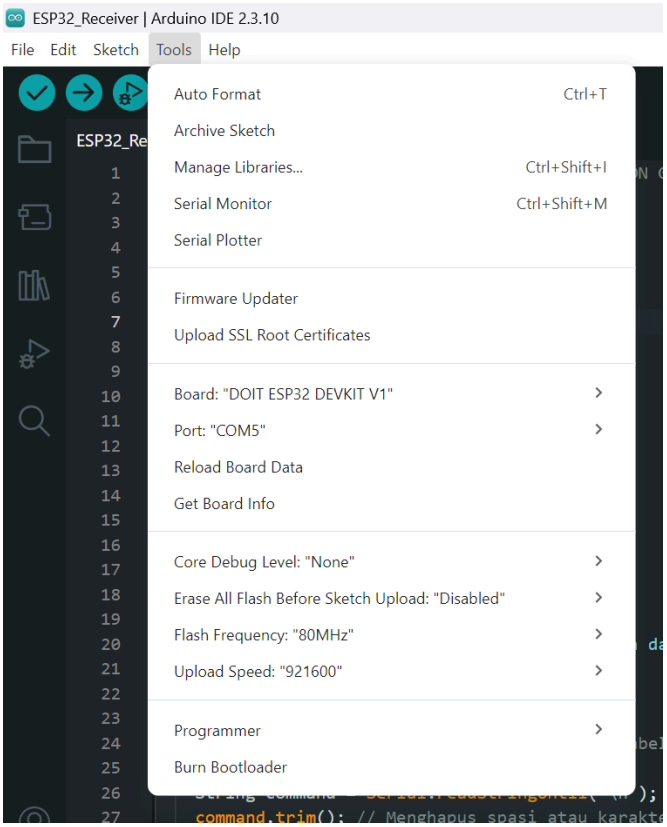
Penjelasan program

1. Inisialisasi & Setup: Program menentukan pin 27 dan 26 sebagai kontrol logika putaran, dan pin 12 sebagai kontrol kecepatan (PWM). API PWM ESP32 yang terbaru diinisialisasi melalui perintah ledcAttach(enable1Pin, freq, resolution). Resolusi diset pada 8-bit, yang artinya kecepatan motor direpresentasikan dalam rentang 0 hingga 255.
2. Fungsi applyMotor(): Ini adalah fungsi *core* yang mentransfer state logis ke hardware. Mengunci motor berputar ke satu arah (motor1Pin1 HIGH, motor1Pin2 LOW), lalu menuliskan nilai variabel global dutyCycle sebagai gelombang PWM ke pin 12. Fungsi ini juga menyalakan LED bila duty cycle lebih dari 0.
3. Pembacaan Serial (Loop): Program menggunakan konsep non-blocking (tidak menggunakan *delay()*). Ia membaca karakter masuk satu persatu secara iteratif dan menyusunnya menjadi inputString.
4. Parsing Instruksi: Ketika pendeteksi spasi baris/enter ditemukan (\n atau \r), pembacaan dihentikan dan string diproses. Apabila string diawali dengan kata "PWM:", maka program akan mengambil angka (substring indeks 4 ke atas) untuk dijadikan *integer* kecepatan. Bila hanya 1 karakter (1, 0, +, -), program

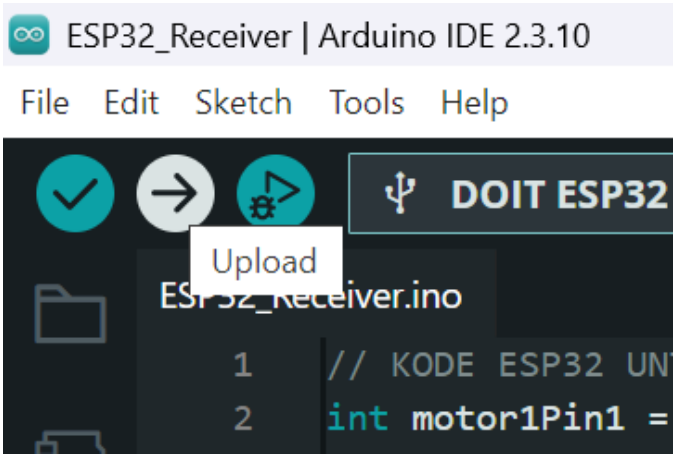
memanggil fungsi yang berkesesuaian. Setelah eksekusi, variabel buffer inputString akan dikosongkan.

F. Hasil Pengujian & Analisis

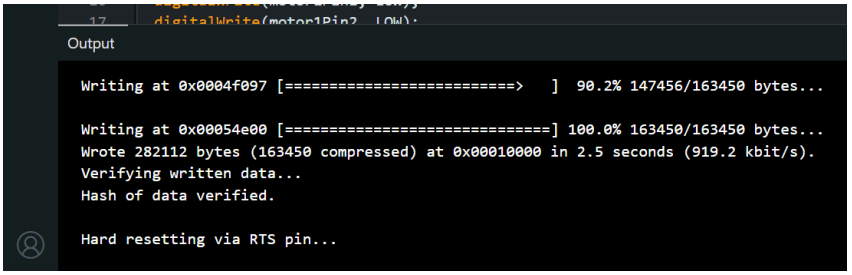
1. Proses Kompilasi dan Upload Program



Pertama, buka menu Tools pada bagian atas Arduino IDE. Pastikan pengaturan Board telah dipilih secara spesifik menjadi "DOIT ESP32 DEVKIT V1" agar compiler mengenali arsitektur perangkat yang digunakan. Setelah itu, pastikan Port (misalnya COM5) sudah terdeteksi dan dicentang. Jika Port tidak muncul, berarti kabel USB belum terpasang dengan benar atau *driver* USB-to-UART belum terinstal di komputer.

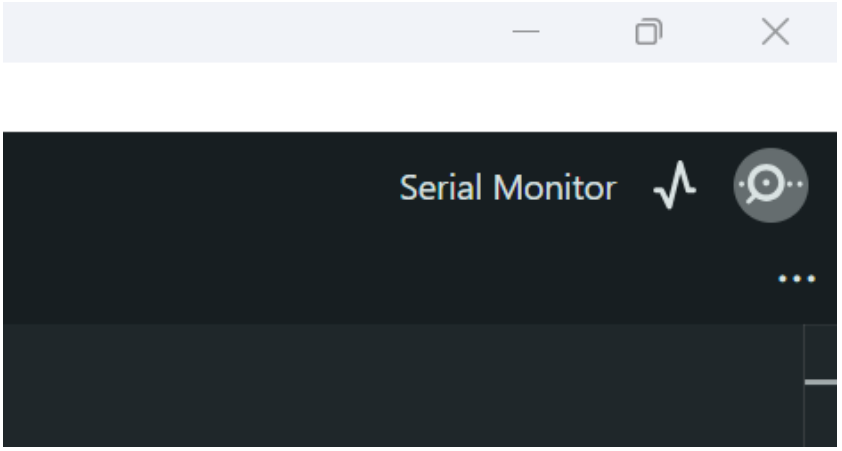


Setelah konfigurasi sesuai, klik tombol Upload (ikon panah ke kanan) yang berada di pojok kiri atas Arduino IDE. Sistem akan melakukan proses kompilasi terlebih dahulu (menerjemahkan bahasa C++ menjadi bahasa mesin). Jika penulisan kode sudah benar dan tidak ada *error*, Arduino IDE akan langsung memulai proses transfer data (*uploading*) ke memori *flash* ESP32.

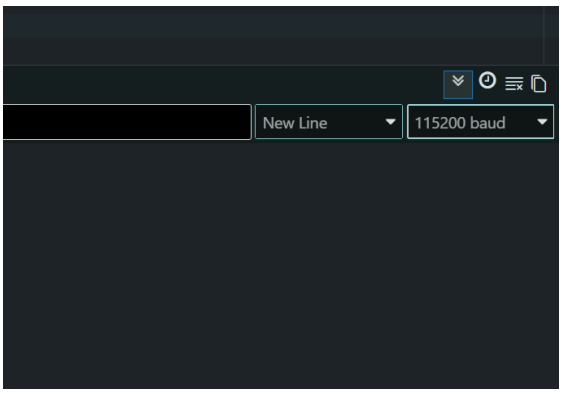


Selama proses *upload* berlangsung, perhatikan jendela *Console/Output* di bagian bawah layar. Akan muncul indikator persentase penulisan memori (Writing at...). Jika proses telah mencapai 100% dan muncul keterangan "Hard resetting via RTS pin...", hal ini menandakan bahwa *firmware* telah berhasil ditanamkan ke dalam ESP32. ESP32 akan secara otomatis melakukan *restart* dan program langsung berjalan saat itu juga.

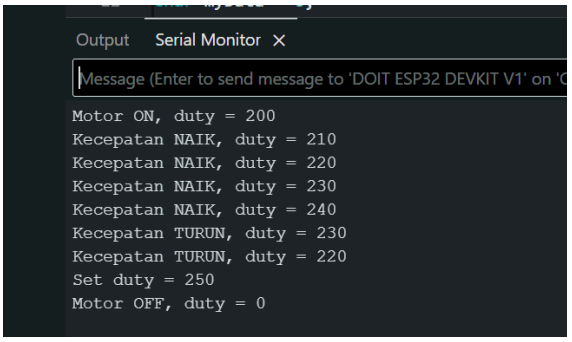
2. Pengujian Perintah via Serial Monitor



Klik ikon Serial Monitor (berbentuk kaca pembesar) yang terletak di sudut kanan atas antarmuka Arduino IDE. Fitur ini berfungsi sebagai terminal untuk melihat pesan yang dikirim oleh ESP32, sekaligus sebagai kolom *chat* untuk mengetikkan perintah teks dari komputer ke ESP32.



Pada pojok kanan bawah jendela Serial Monitor, ubah pengaturan kecepatan komunikasi (*Baud Rate*) menjadi 115200 baud. Angka ini harus sama persis dengan instruksi Serial.begin(115200); pada program. Selain itu, pastikan pengaturan *line ending* diatur ke opsi "New Line". Opsi ini memastikan bahwa setiap kali kita menekan tombol *Enter* di *keyboard*, komputer akan menyisipkan karakter \n tersembunyi yang berguna sebagai penanda bagi ESP32 bahwa satu kalimat perintah telah selesai diketik.



Ketikkan perintah pada kolom teks di bagian atas Serial Monitor, lalu tekan *Enter*. Berdasarkan hasil pengujian pada gambar, sistem merespons instruksi dengan sangat baik:

a. Saat dikirim angka 1, sistem membalas dengan "Motor ON, duty = 200" dan indikator LED menyala, menandakan motor berputar di kecepatan *default*.
b. Saat dikirim simbol + secara berulang, sistem membalas dengan "Kecepatan NAIK" dan nilai *duty cycle* bertambah 10 poin secara bertahap (menjadi 210, 220, 230, 240).
c. Saat dikirim simbol - secara berulang, sistem membalas dengan "Kecepatan TURUN" dan nilai turun 10 poin (menjadi 230, 220).
d. Saat dikirim teks spesifik PWM:250, sistem langsung melompat mengunci kecepatan tinggi dengan balasan "Set duty = 250".
e. Terakhir, saat dikirim angka 0, sistem merespons dengan "Motor OFF, duty = 0", dan tegangan ke motor diputus sepenuhnya.

3. Analisis Keseluruhan

Berdasarkan pengujian yang dilakukan, sistem kontrol Motor DC dan indikator LED menggunakan ESP32 telah beroperasi sesuai instruksi. Penggunaan komunikasi serial terbukti efektif sebagai media interaksi teks antara pengguna dan mikrokontroler. Modulasi PWM dengan resolusi 8-bit pada frekuensi 30.000 Hz mampu memberikan variasi kecepatan putaran motor yang mulus dalam rentang *duty cycle* 0 hingga 255. Selain itu, logika pembatasan nilai (*clamping*) di dalam kode program berfungsi dengan sangat baik; sistem berhasil menahan nilai *duty cycle* agar tidak melebihi batas maksimal (255) saat kecepatan terus dinaikkan, dan mencegah nilai menjadi negatif saat diturunkan. Pembaruan fungsi API PWM pada ESP32 Board v3.0+ (menggunakan ledcAttach dan ledcWrite langsung ke pin) juga secara terbukti menyederhanakan penulisan struktur program.